

# Practical TypeScript for React & WooWoo ERP

A fast-track handbook covering the most essential TypeScript concepts, patterns, React typing, and best practices used day-to-day across our codebase.

🎯 **Target:** Frontend Engineers

⚡ **Codebase:** WooWoo\_Erp (Vite + React + TS)

📅 **Updated:** 2026

## 1 Core TypeScript Types (Everyday Basics)

In TypeScript, we declare variable types to catch bugs before our app even runs in the browser.

```
// 1. Primitive types
const customerName: string = "Khushi Sahu";
const totalAmount: number = 1450.50;
const isPhoneAutoFetched: boolean = true;

// 2. Arrays
const tags: string[] = ["coworking", "vip"];
const prices: number[] = [500, 1000, 2500];

// 3. Null & Undefined (very common when fetching from API or search inputs)
let customerId: string | null = null;    // holds string or null
let discountCode: string | undefined;    // optional or not yet set
```

### ⚠️ Golden Rule: Avoid `any` !

Writing `const data: any` turns off TypeScript completely. If you don't know the exact shape yet, use `unknown` or create a quick custom `type`.

## 2 Defining Objects: `type` vs `interface`

In our ERP, we define the shape of objects like Customers, Spaces, Invoices, and Bookings using `type` or `interface`.

```
// Real example from WooWoo ERP: Customer model
export type Customer = {
  _id: string;
  name: string;
  mobile: string;
  email?: string; // '?' means optional field (can be undefined)
  membershipType?: string | null; // can be string, null, or undefined
  walletBalance: number;
};
```

Use **type** when:

- Defining Union Types (e.g. status lists)
- Component Props & Payloads
- Tuples & Utility type combinations

Use **interface** when:

- You want to extend shapes using **extends**
- Defining large object models for classes/APIs
- Need declaration merging

### 3 Union Types & String Literals (Crucial for ERP Statuses)

Instead of allowing any random string like `"pending"`, `"Pending"`, or `"done"`, TypeScript lets us restrict variables to an exact set of allowed values:

```
// 1. Literal Union for Booking Status
export type SpaceBookingStatus =
  | "Upcoming"
  | "Ongoing"
  | "Draft"
  | "Expired"
  | "Cancelled";

// TypeScript will immediately error if you make a typo!
let currentStatus: SpaceBookingStatus = "Ongoing"; // ✅ Valid
// currentStatus = "On-going";                      // ❌ Error: Type '"On-going"' is not assignable

// 2. Membership Plans Union
export type MembershipTier = "none" | "silver" | "gold" | "platinum";
```

💡 **Why this matters:** Autocomplete in VS Code shows you every valid option instantly with `Ctrl + Space` !

### 4 Typing React Component Props

Every reusable modal, button, and card in WooWoo ERP takes typed props so parents can never pass incorrect values:

```
import React from "react";

// Step 1: Define Props Type
type MembershipBadgeProps = {
  membershipType: string;
  membershipPlanId?: string | null; // optional
  showNone?: boolean; // optional boolean flag
  className?: string;
  onUpgrade?: (planId: string) => void; // callback function
};

// Step 2: Annotate your Component
export const MembershipBadge = ({
  membershipType,
  showNone = false, // default value
  className = "",
  onUpgrade,
}: MembershipBadgeProps) => {
  if (!showNone && membershipType === "none") return null;

  return (
    <span className={`badge ${className}`}>
      {membershipType.toUpperCase()}
    </span>
  );
};
```

## 5 React Hooks with TypeScript (Daily Work)

Hooks work with TypeScript Generics ( `<Type>` ) to know what data they store.

```
// 1. useState with primitives (Inferred automatically)
const [name, setName] = useState(""); // type: string
const [count, setCount] = useState(0); // type: number

// 2. useState with Objects or Null (Must provide <Type | null>)
const [customerId, setCustomerId] = useState<string | null>(null);
const [customer, setCustomer] = useState<Customer | null>(null);

// 3. useState with Array of Objects
const [customers, setCustomers] = useState<Customer[]>([]);

// 4. useRef for DOM Elements
const customerSearchRef = useRef<HTMLDivElement>(null);
const phoneInputRef = useRef<HTMLInputElement>(null);
```

## 6 Event Handling (Forms, Inputs, & Buttons)

One of the most frequent tasks in ERP forms is typing events correctly:

| Event Target       | TypeScript Type                                           | Common Usage                                                     |
|--------------------|-----------------------------------------------------------|------------------------------------------------------------------|
| Text / Phone Input | <code>React.ChangeEvent&lt;HTMLInputElement&gt;</code>    | <code>(e) =&gt; setName(e.target.value)</code>                   |
| Dropdown Select    | <code>React.ChangeEvent&lt;HTMLSelectElement&gt;</code>   | <code>(e) =&gt; setPurpose(e.target.value)</code>                |
| Textarea           | <code>React.ChangeEvent&lt;HTMLTextAreaElement&gt;</code> | <code>(e) =&gt; setNotes(e.target.value)</code>                  |
| Form Submit        | <code>React.FormEvent&lt;HTMLFormElement&gt;</code>       | <code>e.preventDefault();</code><br><code>handleSubmit();</code> |
| Button Click       | <code>React.MouseEvent&lt;HTMLButtonElement&gt;</code>    | <code>(e) =&gt; { e.stopPropagation(); }</code>                  |

```
// Practical Example: Form Input & Submission
const handlePhoneChange = (e: React.ChangeEvent<HTMLInputElement>) => {
  const digitsOnly = e.target.value.replace(/\D/g, "");
  setPhone(digitsOnly);
};

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (!name.trim()) {
    setErrors(prev => ({ ...prev, name: "Name is required" }));
  }
};
```

## 7 Essential TypeScript Utility Types

TypeScript provides built-in helper types to transform existing types effortlessly:

```

type CustomerPayload = {
  name: string;
  mobile: string;
  email: string;
  address: string;
};

// 1. Partial<T> - Makes ALL properties optional (Perfect for edit/patch updates!)
type UpdateCustomerPayload = Partial<CustomerPayload>;
// { name?: string; mobile?: string; email?: string; address?: string; }

// 2. Pick<T, Keys> - Extracts only the selected fields
type CustomerContactInfo = Pick<CustomerPayload, "name" | "mobile">;

// 3. Omit<T, Keys> - Removes specific fields
type CreateWithoutEmail = Omit<CustomerPayload, "email">;

// 4. Record<K, V> - Dictionary / Key-Value map (Used for errors state in forms!)
const errors: Record<string, string> = {
  name: "Name is required",
  phone: "Invalid phone number"
};

```

## 8 Safe Navigation: `?.` and `??`

Prevent `Cannot read properties of undefined` crashes with TypeScript operators:

### Optional Chaining ( `?.` )

Safely accesses deep properties without throwing an error if intermediate keys are null or undefined.

```

customer?.membershipPlanId
initialBooking?.notes?.includes("Purpose:")

```

### Nullish Coalescing ( `??` )

Provides a fallback ONLY when the value is `null` or `undefined` (unlike `||` which also triggers on 0 or empty string).

```

const count = cartCount ?? 0;
const label = title ?? "Untitled";

```

## 9 API Calls & Generics ( `Promise<T>` )

In `apiClient.ts`, we type API responses with Generics so caller components know the exact response format:

```
// Generic API Wrapper
export type ApiResponse<T> = {
  success: boolean;
  message?: string;
  data: T; // T can be anything (Customer[], Booking, Invoice, etc.)
};

// Example API function
export const handleGetCustomers = async (): Promise<Customer[]> => {
  const response = await axiosInstance.get<ApiResponse<Customer[]>>("/customers");
  return response.data.data;
};
```

## 10 TypeScript Debugging Cheatsheet

Common TypeScript error messages and their quick 5-second fixes:

| TypeScript Error                                 | Why it happens                                                                  | How to fix                                                                              |
|--------------------------------------------------|---------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Object is possibly 'null' or 'undefined'         | A variable might be null (e.g. <code>c._id</code> when <code>c</code> is null). | Use optional chaining: <code>c?._id</code> or check with <code>if (c) { ... }</code> .  |
| Property 'xyz' does not exist on type 'abc'      | Typo in property name or field is missing in type definition.                   | Check the type definition and add <code>xyz?: string</code> or fix typo.                |
| Type 'string' is not assignable to type 'number' | Passing a string to a numeric prop (e.g. <code>price="500"</code> ).            | Pass a number with curly braces: <code>price={500}</code> or <code>Number(val)</code> . |
| Binding element 'x' implicitly has an 'any' type | Component props function is missing a type definition.                          | Add props type: <code>(({ x }): { x: string }) =&gt; ...</code>                         |